

Semana 4. GraphQL y federación de servicios

Objetivo: Analizar e implementar modelos avanzados de integración basados en GraphQL, comprendiendo sus fundamentos operativos (queries, mutations y subscriptions), su estructura interna mediante esquemas y resolvers, los beneficios y desafíos frente a REST, y los principios de federación distribuida con Apollo Federation para construir arquitecturas escalables y modulares en ecosistemas de microservicios..

Introducción:

En la cuarta semana del curso se introduce GraphQL como una evolución significativa en la integración de sistemas, especialmente en entornos donde los clientes requieren alto control sobre la forma y cantidad de datos obtenidos desde el backend. A diferencia de REST, que estructura la interacción a través de múltiples endpoints y recursos predefinidos, GraphQL ofrece un enfoque declarativo centrado en un único endpoint, donde el cliente especifica exactamente qué información necesita. Este cambio conceptual habilita mejoras sustanciales en eficiencia, reducción de overfetching y underfetching, y simplificación del consumo desde aplicaciones móviles, SPA y paneles administrativos complejos.

Tras comprender las bases del lenguaje —queries, mutations y subscriptions— la clase profundiza en la forma en que GraphQL modela entidades mediante esquemas tipados y resolvers especializados que vinculan el grafo con microservicios REST y gRPC existentes. Finalmente, se aborda Apollo Federation como una solución moderna para escalar GraphQL en arquitecturas con múltiples dominios, permitiendo que cada microservicio mantenga su propio subgrafo mientras el gateway compone un supergraph unificado. Esta visión federada prepara al estudiante para diseñar sistemas modulares, desacoplados y altamente mantenibles, tal como ocurre en plataformas globales como Amazon, Netflix o Shopify.

4.1. Fundamentos de GraphQL: *queries*, *mutations* y *subscriptions*

GraphQL es un lenguaje de consulta para APIs y un *runtime* para ejecutar esas consultas sobre un esquema fuertemente tipado. Fue creado originalmente por Facebook y hoy es mantenido por la GraphQL Foundation. A diferencia de REST, donde cada recurso se accede a través de múltiples endpoints, GraphQL expone típicamente un **único endpoint** (por ejemplo `/graphql`) a través del cual el cliente envía operaciones estructuradas (*queries*, *mutations* y *subscriptions*) y recibe exactamente los campos que necesita, ni más ni menos.

Este enfoque resulta especialmente útil en arquitecturas de microservicios y en escenarios tipo “Amazon”: catálogos de productos, carritos, pedidos, usuarios, recomendaciones, etc. En lugar de que el frontend tenga que llamar a múltiples APIs REST para construir una sola pantalla, puede enviar una única operación GraphQL que agregue datos de varias entidades relacionadas. En esta sección, nos centraremos en entender los tres tipos de operaciones principales:

- **Queries:** leer datos.
- **Mutations:** modificar datos (crear, actualizar, eliminar).
- **Subscriptions:** recibir actualizaciones en tiempo real.

4.1.1. Queries: lectura declarativa de datos

Una query en GraphQL es una operación de lectura que describe qué datos necesita el cliente. En vez de hacer múltiples llamadas (por ejemplo: /productos, luego /productos/123, luego /reviews?producto=123), el cliente envía una sola query declarativa. El servidor ejecuta la query contra su esquema y devuelve exactamente la forma solicitada de los datos.

Ejemplo sencillo para el contexto de un e-commerce:

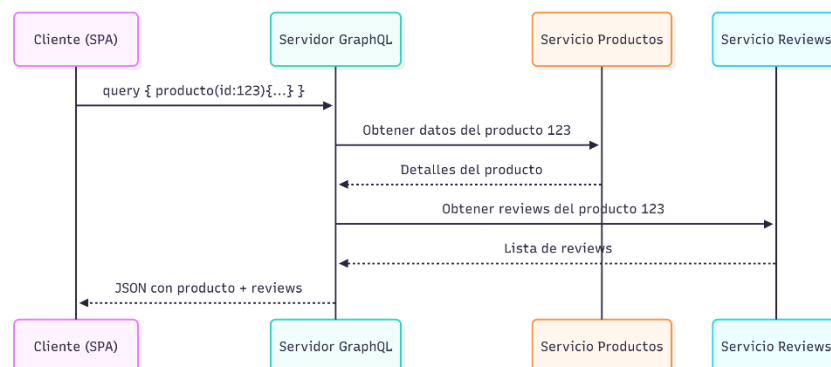
```
query ObtenerProductoYReviews {  
  producto(id: 123) {  
    id  
    nombre  
    precio  
    categoria  
    reviews {  
      usuario {  
        nombre  
      }  
      comentario  
      rating  
    }  
  }  
}
```

El servidor responde con un JSON *en la misma forma* que la query:

```
{  
  "data": {  
    "producto": {  
      "id": 123,  
      "nombre": "Café Orgánico 500g",  
      "precio": 19.9,  
      "categoria": "Bebidas",  
      "reviews": [  
        {  
          "usuario": { "nombre": "Ana" },  
          "comentario": "Excelente sabor",  
          "rating": 5  
        }  
      ]  
    }  
  }  
}
```

Esto reduce el overfetching (traer datos de más) y el underfetching (datos de menos que obligan a más llamadas).

Figura 1. Flujo básico de una query GraphQL



4.1.2. Mutations: escritura y cambios de estado

Las mutations son el mecanismo de GraphQL para escribir datos (crear, actualizar, eliminar). Aunque técnicamente podrían ejecutarse con GET, por convención y buenas prácticas se envían como POST, y semánticamente equivalen a “acciones con efectos de lado”.

Ejemplo de mutation para crear un pedido:

```
mutation CrearPedido($input: PedidoInput!) {  
  crearPedido(input: $input) {  
    id  
    estado  
    total  
    items {  
      producto {  
        nombre  
      }  
      cantidad  
      subtotal  
    }  
  }  
}
```

Con variables:

```
{  
  "input": {  
    "usuarioId": "u-100",  
    "items": [  
      { "productoId": 1, "cantidad": 2 },  
      { "productoId": 3, "cantidad": 1 }  
    ],  
    "direccionEnvio": "Quito, Ecuador"  
  }  
}
```

El servidor ejecuta la lógica de negocio (validar stock, calcular total, registrar el pedido, etc.) y devuelve el nuevo estado del recurso. Nótese que, a diferencia de muchos endpoints REST, GraphQL fomenta devolver inmediatamente el objeto actualizado, lo que simplifica el trabajo del frontend.

Figura 2. Ciclo de vida de una mutation



4.1.3. Subscriptions: actualizaciones en tiempo real

Las subscriptions son el tercer tipo de operación en GraphQL y permiten obtener actualizaciones en tiempo real. A diferencia de las queries (que son de vida corta), una subscription mantiene una conexión activa (frecuentemente WebSocket) y el servidor “empuja” nuevos datos al cliente cuando ocurre un evento (por ejemplo, cambio de estado de un pedido o llegada de una nueva oferta).

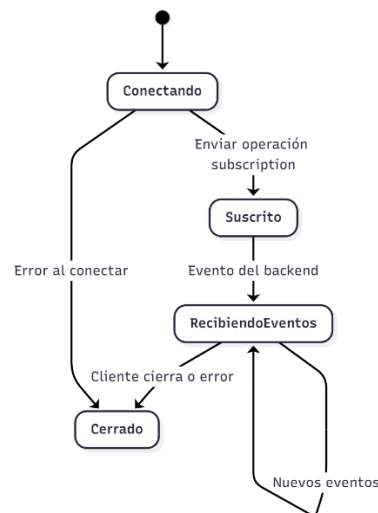
Ejemplo para seguir el estado de un pedido:

```
subscription OnPedidoActualizado($id: ID!) {  
  pedidoActualizado(id: $id) {  
    id  
    estado  
    actualizadoEn  
  }  
}
```

Cada vez que el pedido cambia de estado (CREADO, PAGADO, ENVIADO, ENTREGADO), el servidor empuja un nuevo mensaje al cliente:

```
{
  "data": {
    "pedidoActualizado": {
      "id": "p-5001",
      "estado": "ENVIADO",
      "actualizadoEn": "2025-11-19T15:23:00Z"
    }
  }
}
```

Figura 3. Ciclo de vida de una subscription GraphQL



Este modelo es especialmente útil en una arquitectura tipo Amazon para:

- Actualización de estado de pedidos en tiempo real.
- Notificación de cambios de inventario.
- Ofertas relámpago.
- Notificaciones al panel de administración.

4.1.4. Sintaxis y estructura de las operaciones

Aunque *queries*, *mutations* y *subscriptions* tienen propósitos distintos, comparten una sintaxis muy similar:

- **Operación:** query, mutation o subscription.
- **Nombre opcional:** útil para debugging y seguimiento (query ObtenerProducto).
- **Variables:** definidas en la cabecera ($\{id: ID!\}$).
- **Selección de campos:** árbol de campos anidado que define la forma de la respuesta.

Tabla 1 resume las diferencias principales.

Tabla 1. Comparación de los tipos de operación en GraphQL

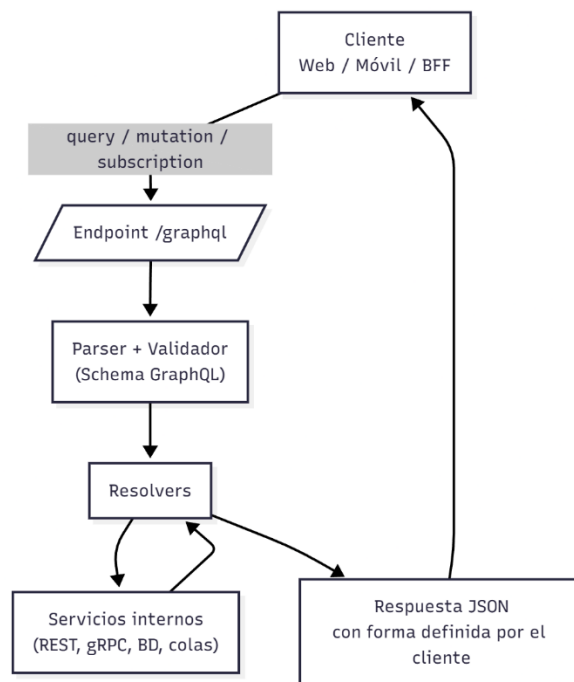
Tipo	Propósito principal	Duración	Ejemplo típico
Query	Lectura de datos	Corta (1 req/resp)	Obtener producto, listado de pedidos
Mutation	Escritura/cambio de estado	Corta (1 req/resp)	Crear pedido, actualizar usuario
Subscription	Actualizaciones en tiempo real	Larga (conexión activa)	Seguir estado de un pedido, notificaciones

4.1.5. Flujo general de una operación GraphQL

Desde la perspectiva de arquitectura de integración, toda operación GraphQL (sea query, mutation o subscription) sigue un flujo similar:

1. El cliente (SPA, app móvil, BFF) envía la operación al endpoint `/graphql`.
2. El servidor GraphQL parsea la operación y la valida contra el esquema (`schema.graphql`).
3. Se ejecutan los *resolvers*, que llaman a servicios REST, gRPC, bases de datos, colas, etc.
4. El servidor construye la respuesta solo con los campos solicitados.
5. En el caso de subscriptions, el flujo sigue activo y se repite el paso 3 cada vez que ocurre un evento relevante.

Figura 4. Arquitectura básica de un servidor GraphQL en integración de sistemas



Este patrón es clave para entender luego la federación de servicios (Apollo Federation), donde múltiples subgráficos se combinan para ofrecer un único “grafo” empresarial.

4.1.6. Buenas prácticas para usar queries, mutations y subscriptions

En el contexto de tu curso y del prototipo tipo Amazon, conviene enfatizar algunas buenas prácticas:

1. Queries idempotentes y cacheables

- Mantener las queries sin efectos de lado para poder cachearlas y reejecutarlas sin riesgo.

2. Mutations con semántica clara de negocio

- Usar nombres orientados al dominio: crearPedido, confirmarPago, cancelarPedido, en lugar de savePedido.

3. Subscriptions solo cuando realmente se necesita tiempo real

- Evitar usar subscriptions para todo; reservarlas para casos donde el *push* desde el servidor aporta valor (estado de pedidos, stock crítico, alertas).

4. Diseñar el esquema pensando en el cliente pero sin romper el dominio

- Equilibrar “esquema orientado a UI” con “esquema orientado a dominio” para no convertir GraphQL en un API extremadamente acoplado a una sola pantalla.

5. Monitorear y limitar operaciones costosas

- Queries muy profundas o con demasiados campos pueden degradar el backend; es importante aplicar *query complexity analysis* y límites de profundidad.

4.1.7. Conexión con el proyecto del curso

Para el proyecto integrador tipo Amazon, el uso de GraphQL puede introducirse de forma incremental:

- En etapas iniciales, usar queries y mutations para construir un BFF GraphQL sobre microservicios REST y gRPC existentes.
- Más adelante, incorporar subscriptions para el panel de administración (actualización de inventarios en tiempo real, estado de pedidos, métricas internas).
- Esta base conceptual será fundamental para la sección 4.4, donde se analizará Apollo Federation como mecanismo para combinar múltiples esquemas en un solo grafo corporativo.

4.2. Esquemas, resolvers y tipos en GraphQL

GraphQL se basa en un concepto central: el esquema.

El esquema define *qué* datos están disponibles, *cómo* se relacionan y *qué operaciones pueden ejecutarse*. A diferencia de REST, donde la estructura de la API se deduce de la combinación de rutas y controladores, en GraphQL todo el diseño está explícito en el schema, que actúa como un contrato entre el frontend y el backend.

Los elementos fundamentales del esquema son:

- **Tipos (types):** describen entidades, estructuras y valores.
- **Resolvers:** funciones que implementan cómo obtener los datos.
- **Schema:** conjunto de tipos + operaciones (Query, Mutation, Subscription).

Cada operación ejecutada por el cliente se valida contra el esquema. Cualquier campo que no esté definido es inválido, lo que permite un nivel de robustez similar al de Proto3 en gRPC y mucho mayor que el de REST.

4.2.1. El esquema (*schema*) como contrato entre cliente y servidor

El schema de GraphQL se escribe generalmente en *SDL (Schema Definition Language)*, un lenguaje declarativo legible y simple.

Por ejemplo, un esquema mínimo para un e-commerce puede verse así:

```
type Producto {
  id: ID!
  nombre: String!
  precio: Float!
  categoria: String!
  stock: Int!
}

type Query {
  producto(id: ID!): Producto
  productos(categoria: String!): [Producto!]!
}

type Mutation {
  crearProducto(nombre: String!, precio: Float!, categoria: String!): Producto!
}

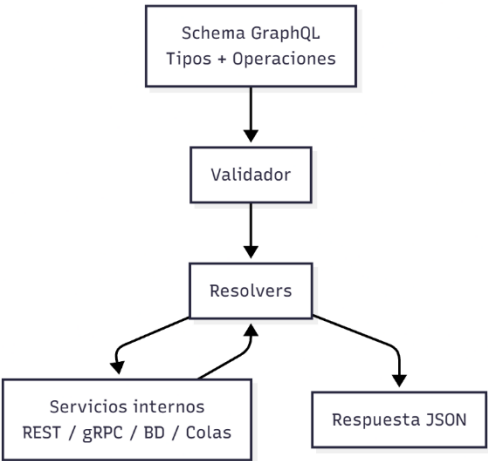
type Subscription {
  productoActualizado(id: ID!): Producto!
}
```

Este esquema describe:

- Un tipo Producto.
- Dos queries (producto y productos).
- Una mutation (crearProducto).
- Una subscription (productoActualizado).

El cliente solo puede solicitar los campos definidos en este esquema.

Figura 5. Relación conceptual del esquema en GraphQL



El esquema no contiene lógica de negocio.
Es únicamente una descripción formal de las posibilidades.

4.2.2. Tipos en GraphQL: la base del esquema

Los tipos son el corazón del lenguaje. GraphQL proporciona varios tipos:

1. Tipos escalares (Scalars)

Los escalares representan valores atómicos:

Escalar	Descripción
Int	Entero
Float	Número decimal
String	Cadena de texto
Boolean	Verdadero/Falso
ID	Identificador único

Ejemplo:

```
type Usuario {
  id: ID!
  nombre: String!
  activo: Boolean!
}
```

2. Objetos (Object types)

Los objetos representan entidades del dominio (ej. productos, pedidos).

```
type Pedido {
  id: ID!
  total: Float!
  usuario: Usuario!
  items: [ItemPedido!]!
}
```

3. Enums

Tipos enumerados ideales para estados, categorías u opciones.

```
enum EstadoPedido {
  CREADO
  PAGADO
  EN_PROCESO
  ENVIADO
  ENTREGADO
}
```

4. Input Types

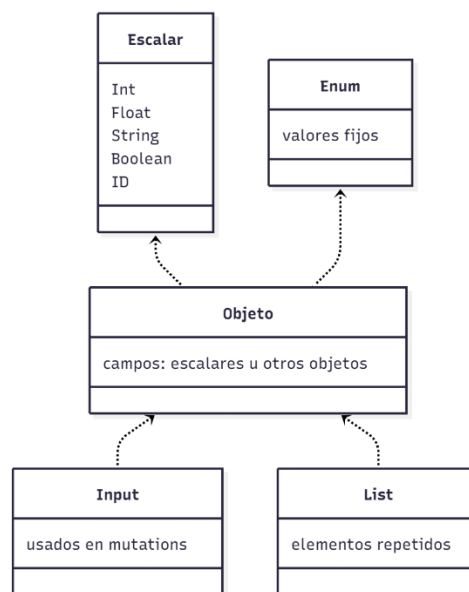
Se usan en *mutations* para envío estructurado.

```
input PedidoInput {
  usuarioid: ID!
  items: [ItemInput!]!
  direccionEnvio: String!
}
```

5. Listas y no-nulos

- [Producto] = lista de productos.
- Producto! = siempre debe haber un producto.
- [Producto!]! = lista no nula de productos no nulos.

Figura 6. Tipos principales de GraphQL



4.2.3. Los resolvers: funciones que obtienen los datos

Aunque el esquema define *qué* datos existen, los resolvers definen *cómo* obtenerlos.

Un resolver es una función que calcula el valor de un campo específico del esquema.

Ejemplo basado en Node.js:

```
const resolvers = {
  Query: {
    producto: (_, { id }, { dataSources }) =>
      dataSources.productosAPI.obtenerProducto(id),

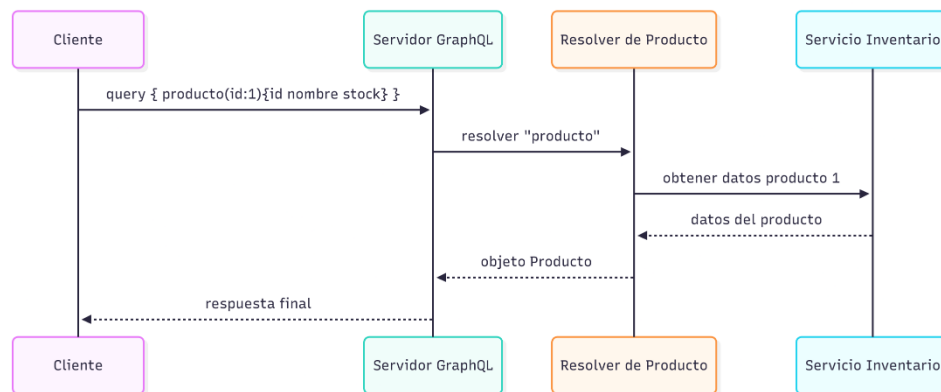
    productos: (_, { categoria }, { dataSources }) =>
      dataSources.productosAPI.listarProductos(categoria)
  },

  Product: {
    reviews: (producto, _, { dataSources }) =>
      dataSources.reviewsAPI.obtenerReviews(producto.id)
  }
};
```

Esto demuestra dos cosas:

1. Cada campo tiene potencialmente su propio resolver.
2. GraphQL permite composición de datos (ej. Producto → reviews).

Figura 7. Ciclo resolver por campo



Cada nivel del árbol de la query se resuelve recursivamente de esta forma.

4.2.4. La relación entre tipos y resolvers

El resolver coincide en nombre con el tipo y el campo.

Ejemplo:

```
type Producto {
  id: ID!
  nombre: String!
  stock: Int!
}

Resolver correspondiente:
Producto: {
  stock: (producto, args, context) => {
    return context.inventarioAPI.obtenerStock(producto.id);
  }
}
```

Si no se define un resolver, GraphQL usa un resolver por defecto que simplemente toma el valor del campo en el objeto fuente.

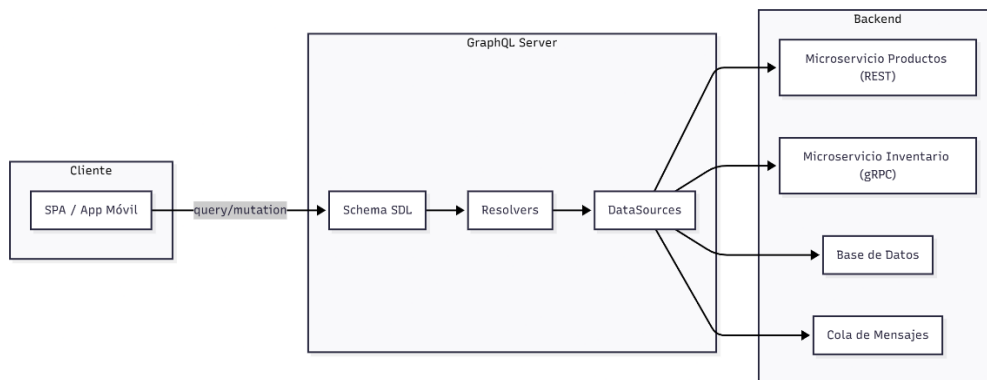
4.2.5. Tabla: diferencias entre esquema y resolvers

Elemento	Qué es	Función	Se parece a...
<i>Schema</i>	Declaración formal del API	Describe datos y operaciones	Contratos OpenAPI o .proto
<i>Types</i>	Componentes del schema	Estructuran entidades y relaciones	DTOs, models
<i>Resolvers</i>	Funciones de ejecución	Obtienen datos reales	Controladores o handlers
<i>Fields</i>	Atributos del esquema	Definen el árbol de respuesta	Propiedades de objetos

4.2.6. Arquitectura interna de GraphQL en un microservicio

En un sistema moderno, el servidor GraphQL suele actuar como un BFF (Backend for Frontend) o como un Gateway de datos, agregando varios microservicios REST o gRPC.

Figura 8. Arquitectura interna de un servidor GraphQL



Esto permite integrar REST + gRPC + colas en un solo endpoint.

4.2.7. Errores comunes en esquemas y resolvers

1. Sobreexposición de datos
Algunos campos deben mantenerse privados (medio de pago, tokens, etc.).
2. Resolvers demasiado pesados
Si cada resolver hace una llamada a BD, puede generar *N+1 queries*.
3. Esquema acoplado al frontend
No debe modelarse “por pantalla”, sino por dominio.
4. Falta de paginación
Listar todo (ej. 5000 productos) es un problema.
5. Uso excesivo de campos opcionales
Conviene usar ! (no-null) cuando haya garantías de negocio.

4.2.8. Buenas prácticas en esquemas y resolvers

- Definir un esquema que refleje el dominio.

- Usar resolvers pequeños y especializados.
- Implementar caché por campo.
- Aplicar DataLoader para evitar problemas N+1.
- Mantener el esquema versionado y documentado.
- Revisar impacto de cada campo en la carga del sistema.

4.2.9. Conclusiones

Los esquemas y resolvers son la base operativa de GraphQL. El esquema define la “forma” del mundo; los resolvers implementan su “realidad”.

Comprender esta separación y su interacción permite:

- Integrar múltiples microservicios bajo un solo endpoint.
- Optimizar el consumo desde el frontend.
- Evitar errores típicos de APIs REST distribuidas.
- Preparar la arquitectura para federación (tema de 4.4).

El dominio bien modelado en un *schema* y el correcto diseño de resolvers constituyen el pilar fundamental para construir APIs GraphQL empresariales modernas, escalables y mantenibles.

4.3. GraphQL vs REST: ventajas y retos

GraphQL y REST son dos enfoques distintos para diseñar APIs modernas. REST, basado en recursos accesibles mediante HTTP, se ha convertido en un estándar de facto para servicios web y APIs públicas durante más de 15 años. Por su parte, GraphQL, creado por Facebook en 2012 y liberado en 2015, propone un modelo basado en consultas declarativas y esquemas fuertemente tipados que permiten obtener exactamente los datos necesarios, agregando flexibilidad y eficiencia en muchas situaciones del frontend moderno (Facebook GraphQL, 2015).

Comprender las diferencias entre REST y GraphQL es esencial para construir arquitecturas híbridas eficientes donde cada tecnología aporta ventajas complementarias. En sistemas complejos tipo Amazon —con catálogos masivos, múltiples microservicios, paneles administrativos avanzados, apps móviles y SPA— es común combinar ambos enfoques: REST para operaciones públicas estables y GraphQL para queries dinámicas de composición compleja.

4.3.1. Arquitectura conceptual: recursos vs grafo

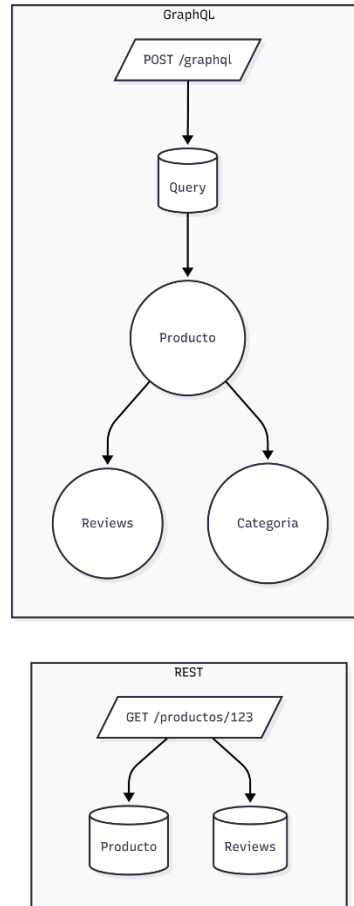
REST organiza la API alrededor de recursos, accesibles a través de URIs. Por ejemplo:

- /productos
- /productos/123
- /productos/123/reviews

Cada endpoint devuelve una estructura fija, con verbos HTTP como GET, POST, PUT y DELETE.

GraphQL organiza la API alrededor de un grafo de entidades relacionadas, expuesto mediante un único endpoint (/graphql). El cliente describe exactamente qué campos y relaciones desea obtener.

Figura 9. REST vs GraphQL — Modelo conceptual



4.3.2. Problemas clásicos de REST: overfetching y underfetching

REST enfrenta un problema frecuente:

Overfetching (traer más datos de los que se necesitan)

Ejemplo: un endpoint /productos/123 devuelve 40 atributos, pero la pantalla del usuario solo necesita tres.

Underfetching (traer menos datos de los necesarios)

El cliente debe hacer múltiples llamadas:

1. /productos/123
2. /productos/123/reviews
3. /usuarios/45

En GraphQL, esto se resuelve con:

```
query {
  producto(id:123){
    id
    nombre
    precio
    reviews { comentario usuario { nombre } }
  }
}
```

Un solo *round trip*, sin exceso de datos.

Tabla 2. Comparación entre REST y GraphQL (nivel técnico-operativo)

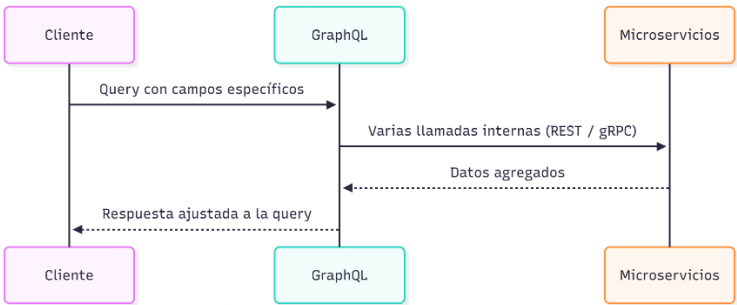
Característica	REST	GraphQL
Endpoint único	✗ Muchos endpoints	✓ Un solo endpoint
Overfetching	Frecuente	Mínimo
Underfetching	Frecuente	Resolución de múltiples entidades en una query
Estructura de la respuesta	Fija	Determinada por el cliente
Tipado fuerte	✗ No nativo	✓ Nativo (SDL)
Documentación	Swagger/OpenAPI	Autogenerada desde el esquema
Caching	Sencillo (HTTP)	Complejo
Tiempo real	Vía WebSockets/SSE	✓ Nativo mediante Subscriptions
Seguridad	Basada en recursos	Basada en resolver/campo
Performance	Bueno	Excelente en composición, peor para cargas masivas

4.3.3. Ventajas de GraphQL sobre REST

Ventaja 1: Consultas precisas y estructuradas

GraphQL permite declarar exactamente qué datos se requieren. Esto reduce la transferencia de datos y la cantidad de llamadas al backend.

Figura 10. Precisión de GraphQL



Ventaja 2: Composición de datos multi-servicio

En arquitecturas modernas, un solo componente del frontend puede requerir datos de varios microservicios:

- Catálogo (REST)
- Inventario (gRPC)
- Recomendación (AI)
- Reviews (REST)
- Precios dinámicos (microservicio interno)

GraphQL actúa como un aggregator layer.

Ventaja 3: Tipado fuerte y validación automática

Un esquema GraphQL define:

- Tipos
- Relaciones
- Arguments
- Queries/Mutations
- Subscriptions

Esto evita inconsistencias típicas en REST donde:

- la documentación puede desactualizarse,
- los DTOs pueden no coincidir con la estructura real,
- respuestas pueden variar según el backend.

Ventaja 4: Evolución del API sin versionamiento explícito

En REST, agregar un campo puede requerir v1, v2, v3.

En GraphQL:

- Nuevos campos → no rompen clientes
- Campos antiguos → pueden marcarse como deprecated

```
type Producto {  
  id: ID!  
  nombre: String!  
  precio: Float!  
  stock: Int! @deprecated(reason: "Usar inventario.stockDisponible")  
}
```

Ventaja 5: Subscriptions en tiempo real

REST no soporta tiempo real nativo; necesita WebSockets/SSE.

GraphQL sí:

- Actualización de inventarios

- Cambios de estado del pedido
- Chat interno
- Alertas administrativas

4.3.4. Retos de GraphQL frente a REST

GraphQL no es una bala de plata. Tiene desafíos importantes, especialmente para backend.

Reto 1: Caching complejo

REST se beneficia de:

- Cache-Control
- ETag
- CDN estándar

GraphQL requiere:

- caching por query
- caching por resolver
- análisis de complejidad
- herramientas especializadas (Apollo Cache, Redis, DataLoader)

Reto 2: Riesgo de N+1 queries

Un resolver mal diseñado puede activar múltiples llamadas:

Ejemplo:

```
{
  productos {
    id
    nombre
    reviews {
      comentario
    }
  }
}
```

Esto podría provocar:

1 llamada para productos

- N llamadas para reviews

Soluciones:

- DataLoader
- Batching
- Caché por entidad

Reto 3: Seguridad granular

REST → seguridad por endpoint y método

GraphQL → seguridad por campo y resolver

Nuevas complejidades:

- Autorización campo a campo
- Query cost analysis
- Profundidad máxima de queries

Reto 4: Sobrecarga en resolvers

GraphQL puede causar:

- alta concurrencia en resolvers
- saturación del backend por queries profundas
- problemas de introspección excesiva

Reto 5: Complejidad operativa

REST es simple:

- Fácil de monitorear
- Fácil de registrar
- Fácil de tracear (correlation ID por endpoint)

GraphQL requiere:

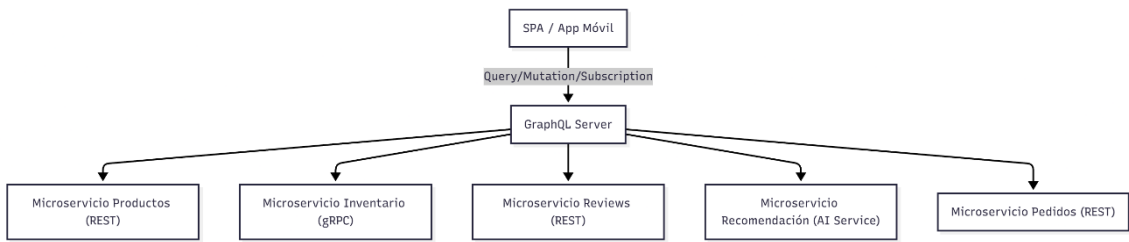
- tracing por campo
- logs por resolver
- herramientas como Apollo Studio
- métricas especializadas

4.3.5. Comparación profunda para prototipo tipo Amazon

Tabla 3. REST vs GraphQL en módulos específicos del prototipo

Módulo	REST	GraphQL	Recomendación
Productos	Eficiente para GET simples	Excelente para búsquedas complejas	GraphQL
Inventario	Consistencia estricta	Difícil integrar stock en consultas complejas	REST/gRPC
Pedidos	Operaciones claras	Permite composición (pedido+items+estado)	GraphQL
Recomendación	No apto	Ideal (datos dinámicos por usuario)	GraphQL
Carrito	CRUD simple	Sin ventaja significativa	REST
Panel Admin	Necesita múltiples llamadas	GraphQL optimiza UI complejas	GraphQL
Streaming	No nativo	Subscriptions nativas	GraphQL
API pública	Estable y simple	Muy flexible pero costoso	REST

4.3.6. Diagrama de arquitectura híbrida REST + GraphQL



Este modelo híbrido es ampliamente usado por:

- Amazon
- Netflix
- Airbnb
- Shopify

4.3.7. Conclusiones de GraphQL vs REST

- GraphQL supera a REST en consultas complejas, composición de datos y evolución del API.
- REST sigue siendo superior para caching, estabilidad, simplicidad, control de acceso granular y operaciones CRUD clásicas.
- Ambos se complementan, no compiten.
- Para el proyecto tipo Amazon, la arquitectura ideal es híbrida:
 - REST/gRPC para backend interno crítico

- GraphQL como capa de agregación para el frontend

4.4. Arquitectura federada (Apollo Federation)

La evolución natural de GraphQL en arquitecturas empresariales complejas es la federación de servicios. A diferencia de un servidor GraphQL monolítico, donde un único esquema concentra toda la lógica, Apollo Federation permite dividir un grafo empresarial en múltiples subgrafos independientes que representan dominios específicos (productos, usuarios, pedidos, inventario, recomendaciones, etc.). Cada subgrafo es mantenido por su propio equipo, con sus propios ciclos de vida, su propio esquema y su propia lógica interna.

Apollo Federation se ha convertido en el estándar moderno para integrar GraphQL dentro de ecosistemas de microservicios, permitiendo que cada servicio aporte solo una parte del grafo global, mientras un supergraph —orquestado mediante un gateway— compone los subgrafos en tiempo de ejecución. En organizaciones como Netflix, Walmart o Airbnb, este enfoque permite coordinar cientos de equipos en paralelo sin que el crecimiento afecte la cohesión del sistema. Para el prototipo tipo Amazon de este curso, Federation permite modelar dominios independientes —catálogo, inventario, reviews, pedidos, usuarios— que se combinan en un único endpoint GraphQL para el frontend.

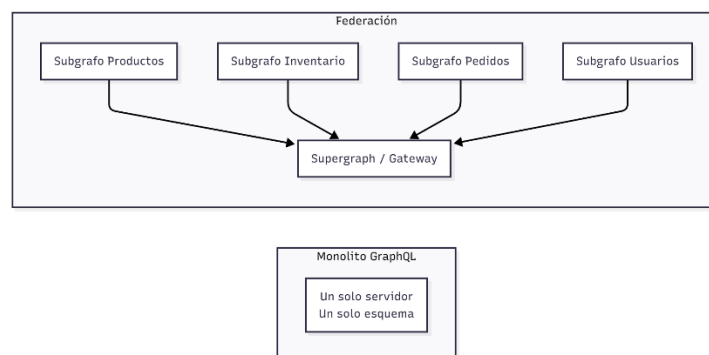
4.4.1. Problema: GraphQL monolítico no escala en microservicios

Un solo servidor GraphQL tiene limitaciones severas:

1. **Cuello de botella del equipo.** Todos los equipos deben coordinar cambios en un mismo esquema grande.
2. **Acoplamiento fuerte.** Un cambio en un campo o tipo afecta a todas las áreas.
3. **Riesgo operacional.** Si el servidor GraphQL cae, cae todo el API.
4. **Problemas de gobernanza.** Se vuelve imposible controlar quién mantiene qué parte del esquema.
5. **Dificultad para escalar equipos.** Un único repo, un único deploy, un único pipeline.

En contraste, el modelo federado permite que cada dominio mantenga *su propio grafo*.

Figura 11. Comparación arquitectónica: Monolito GraphQL vs Federación



4.4.2. ¿Qué es Apollo Federation?

Apollo Federation es un modelo donde:

- Cada microservicio define su propio esquema GraphQL parcial.

- Los esquemas se amplían entre sí mediante `extend`.
- El gateway GraphQL compone un supergraph dinámico.
- Las relaciones entre entidades se definen mediante `keys`.
- Los resolvers remotos son manejados por el gateway.

Ejemplo conceptual:

- El microservicio **Productos** define `Producto`.
- El microservicio **Inventario** extiende `Producto` agregando el campo `stock`.
- El microservicio **Reviews** extiende `Producto` con `reviews`.
- El gateway construye un grafo unificado:

```
type Producto {
  id: ID!
  nombre: String!
  precio: Float!
  stock: Int!
  reviews: [Review!]!
}
```

4.4.3. Conceptos clave de la Federación

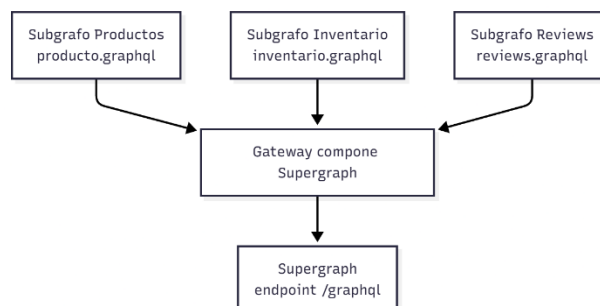
- 1. Subgraphs (subgrafos)** Cada microservicio mantiene su propio grafo.
- 2. Supergraph** El grafo completo, resultado de unir todos los subgrafos.
- 3. Gateway** El servicio que orquesta, valida y resuelve delegaciones entre subgrafos.
- 4. Keys** Identificadores que definen la “entidad federada”.

```
type Producto @key(fields: "id") {
  id: ID!
}
```

5. Extensiones (`extend`) Permiten agregar campos a un tipo definido en otro servicio:

```
extend type Producto @key(fields: "id") {
  id: ID! @external
  stock: Int!
}
```

Figura 12. Flujo de composición del Supergraph



4.4.4. Ventajas de Apollo Federation

Ventaja 1: Independencia de equipos

Cada equipo administra su propio esquema, resolvers y lógica interna.

Ventaja 2: Escalabilidad organizacional

Se pueden tener decenas o cientos de subgrafos.

Ventaja 3: Evolución flexible

Cambiar campos en un subgrafo no rompe a los demás.

Ventaja 4: Autonomía tecnológica

Cada microservicio puede ser:

- Node.js
- Go
- Python
- Java
- .NET
- Rust

Ventaja 5: Superación del acoplamiento REST

GraphQL federado puede unir REST, gRPC, colas y bases de datos.

4.4.5. Ejemplo aplicado a un prototipo tipo Amazon

Subgrafo 1: Productos

```
type Producto @key(fields: "id") {  
  id: ID!  
  nombre: String!  
  precio: Float!  
}
```

Subgrafo 2: Inventario

```
extend type Producto @key(fields: "id") {  
  id: ID! @external  
  stock: Int!  
}
```

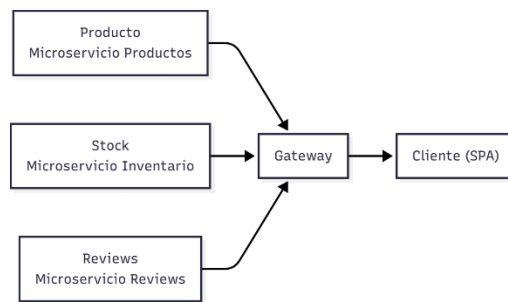
Subgrafo 3: Reviews

```
extend type Producto @key(fields: "id") {  
  id: ID! @external  
  reviews: [Review!]!  
}
```

Resultado final en el Supergraph

```
type Producto {  
  id: ID!  
  nombre: String!  
  precio: Float!  
  stock: Int!  
  reviews: [Review!]!  
}
```

Figura 13. Federación aplicada al dominio Producto



4.4.6. Resolución distribuida de campos

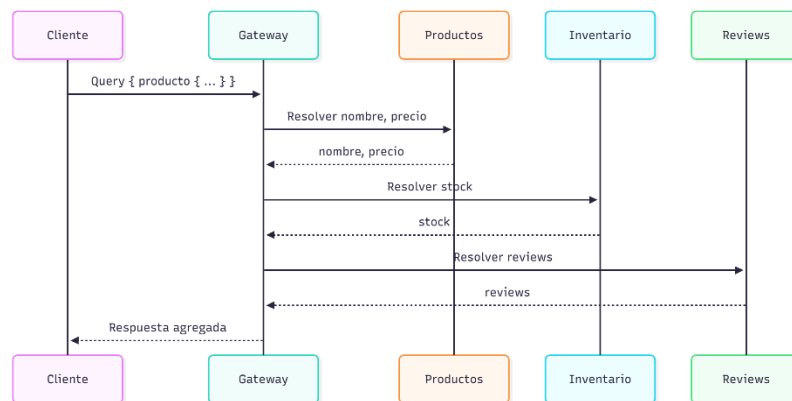
Cuando el cliente hace una query como:

```
query {  
  producto(id: 100) {  
    nombre  
    precio  
    stock  
    reviews { comentario rating }  
  }  
}
```

El gateway divide la query:

1. **Productos** resuelve nombre y precio.
2. **Inventario** resuelve stock.
3. **Reviews** resuelve reviews.

Figura 14. Flujo distribuido de resoluciones



4.4.7. Retos de Apollo Federation

1. Complejidad operativa

Implica:

- Múltiples subgrafos
- Múltiples pipelines
- Múltiples repositorios

2. Depuración distribuida

El tracing debe recorrer varios microservicios.

3. Gobernanza del esquema

Requiere reglas estrictas:

- nombres
- descripciones
- anotaciones
- versionado
- owners

4. Latencia acumulada

Cada campo federado puede generar un *round trip* interno.

Se resuelve con:

- caching
- batching
- optimización en el gateway
- prefetch

4.4.8. Tabla: Comparación entre un servidor GraphQL monolítico y un sistema federado

Aspecto	Monolito GraphQL	Apollo Federation
Escalabilidad	Baja	Muy alta
Arquitectura	Centralizada	Distribuida
Equipos	Dependientes	Autónomos
Versionamiento	Complejo	Por subgrafo
Latencia	Inferior	Puede aumentar si no se optimiza
Gobernanza	Difícil	Estructurada
Adopción empresarial	Media	Alta

4.4.9. Caso real: Amazon y federación por dominio

Aunque Amazon no usa GraphQL para toda su capa de frontend, sí usa modelos federados por dominio con servicios fuertemente especializados (Amazon Builders' Library, 2021).

Dominios típicos:

- Catálogo
- Inventario

- Fulfillment
- Precios dinámicos
- Recomendación (ML)
- Logística
- Estado de pedido

Cada equipo aporta capacidades autónomas y API-first. En un contexto académico, federación GraphQL permite simular este modelo con claridad.

4.4.10. Conclusión

La federación GraphQL permite que los microservicios mantengan independencia total, mientras que el frontend recibe una experiencia unificada mediante un único endpoint. Esto habilita:

- escalabilidad organizacional,
- autonomía técnica,
- seguridad granular,
- composición distribuida,
- evolución del API sin romper clientes.

En el prototipo tipo Amazon, Federation permite que cada equipo desarrolle su propio dominio y que el gateway GraphQL actúe como un cerebro integrador entre REST, gRPC y servicios internos distribuidos.

4.5. Integración híbrida REST + GraphQL

En arquitecturas empresariales modernas, la pregunta no es si se debe utilizar REST o GraphQL, sino cómo combinarlos estratégicamente para obtener el máximo beneficio de cada enfoque. REST continúa siendo una tecnología estable, ampliamente adoptada y compatible con herramientas estándar; GraphQL, por su parte, proporciona flexibilidad, agregación de datos y eficiencia para aplicaciones complejas en frontend.

La integración híbrida REST + GraphQL posibilita un modelo en el que:

- **REST** cumple funciones de estabilidad, estandarización, caching, auditoría y compatibilidad a largo plazo.
- **GraphQL** actúa como *capa de orquestación*, agregando múltiples fuentes y entregando al frontend consultas altamente personalizables.

Este patrón es utilizado por empresas como AWS, Twitter, Shopify, Airbnb, Netflix y Mercado Libre, que mantienen miles de microservicios REST y gRPC, pero exponen un GraphQL API Gateway como interfaz principal para clientes web y móviles.

Para el prototipo tipo Amazon de esta asignatura, este enfoque híbrido permite integrar microservicios REST existentes (catálogo, inventario, pagos, pedidos) con un aggregator GraphQL que ofrece un endpoint unificado para el frontend.

4.5.1. Motivación para integrar REST y GraphQL

REST sigue siendo insuperable en varias áreas:

- Caching HTTP estándar
- Simetría con infraestructura existente
- Simplicidad conceptual
- Soporte ubicuo (pasarelas, proxies, CDNs, WAF)
- Compatibilidad con herramientas como Postman, curl, Swagger, APIM, Apigee

Sin embargo, el frontend moderno enfrenta necesidades que REST dificulta cubrir:

- Obtener datos de múltiples servicios en un solo request
- Reducir el overfetching de grandes payloads JSON
- Evitar múltiples round-trips desde SPA/móvil
- Unificar datos heterogéneos (REST + gRPC + colas)

GraphQL resuelve estos problemas, pero carece de ciertas ventajas operativas que REST maneja bien.

Figura 15. Comparación estratégica REST vs GraphQL

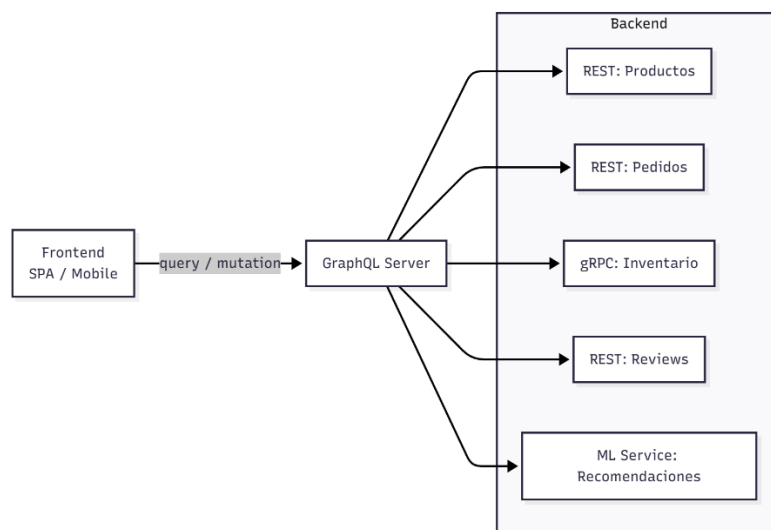


Así, una integración híbrida aprovecha ambos mundos.

4.5.2. Patrón arquitectónico: GraphQL como Data Orchestrator

El patrón dominante en la industria consiste en usar GraphQL como capa de orquestación (BFF o API Gateway inteligente), mientras que los microservicios se mantienen en REST y gRPC.

Figura 16. Arquitectura híbrida (GraphQL como aggregator)



Aquí, GraphQL:

- recibe una operación del cliente,
- resuelve campos,
- delega consultas a diversos servicios,
- y devuelve un JSON con la forma requerida.

El frontend solo llama a un endpoint: **/graphql**.

4.5.3. Casos de uso donde GraphQL complementa a REST

Caso 1: Pantallas complejas en el frontend

Ejemplo: la página de producto de un e-commerce requiere:

- producto
- inventario
- reviews
- precio dinámico
- recomendaciones
- políticas de envío

Con REST → 6 llamados

Con GraphQL → 1 query

Caso 2: Panel administrativo

Los paneles internos suelen tener:

- tablas con filtros
- paneles de métricas
- detalles de pedido
- cargas de trabajo
- dashboards en tiempo real

GraphQL simplifica la extracción de datos agregados.

Caso 3: Aplicaciones móviles

Los límites de ancho de banda hacen que GraphQL sea ideal al evitar overfetching.

Caso 4: Proyectos con microservicios heterogéneos

Si un sistema tiene:

- REST
- gRPC

- Kafka
- Redis
- PostgreSQL
- ML services

GraphQL permite unificar todo bajo una única interfaz pública.

4.5.4. Patrones técnicos de integración híbrida

Patrón 1. GraphQL sobre REST

Los resolvers llaman a microservicios REST existentes.

Ejemplo en Node.js:

```
Query: {
  producto: (_, { id }, { dataSources }) =>
    dataSources.productosAPI.get(`/productos/${id}`)
}
```

Patrón 2. GraphQL + gRPC

Resolvers llaman a servicios internos gRPC:

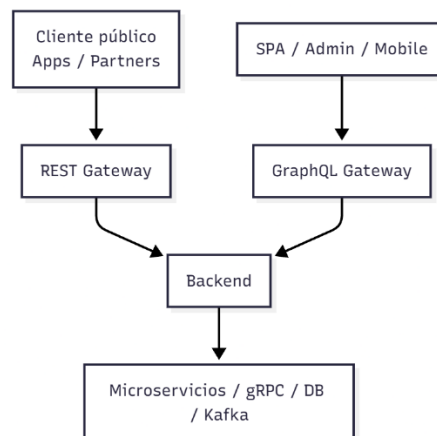
```
Producto: {
  stock: (producto, _, { inventarioClient }) =>
    inventarioClient.getStock({ id: producto.id })
}
```

Patrón 3. Hybrid Gateway (REST + GraphQL)

Algunas empresas exponen:

- /api/v1/ para REST público
- /graphql para consumo interno o SPA
- /events para streaming de eventos

Figura 17. Gateway híbrido REST + GraphQL



4.5.5. Tabla: Cuándo usar REST y cuándo usar GraphQL

Necesidad	REST	GraphQL	Preferencia
Operaciones CRUD simples	✓	✓	REST
Consultas complejas / agregación	✗	✓	GraphQL
Datos en tiempo real	✗	✓ (subscriptions)	GraphQL
Integración con CDN	✓	✗	REST
Documentación legible	✓ Swagger	✓ SDL	Ambos
Bajo consumo en móviles	✗	✓	GraphQL
Compatibilidad con terceros	✓	✗	REST
Seguridad granular	✓	Requiere esfuerzo	REST
Flexibilidad en UI	✗	✓	GraphQL

4.5.6. Estrategia para el prototipo tipo Amazon

REST para:

- carrito
- pagos
- inventario interno
- órdenes (parte transaccional)
- gRPC para sincronización

GraphQL para:

- página de detalle del producto
- homepage del ecommerce
- panel interno administrativo
- dashboard de métricas
- búsquedas avanzadas
- recomendaciones
- composición de datos

4.5.7. Ejemplo completo: GraphQL integrando REST y gRPC

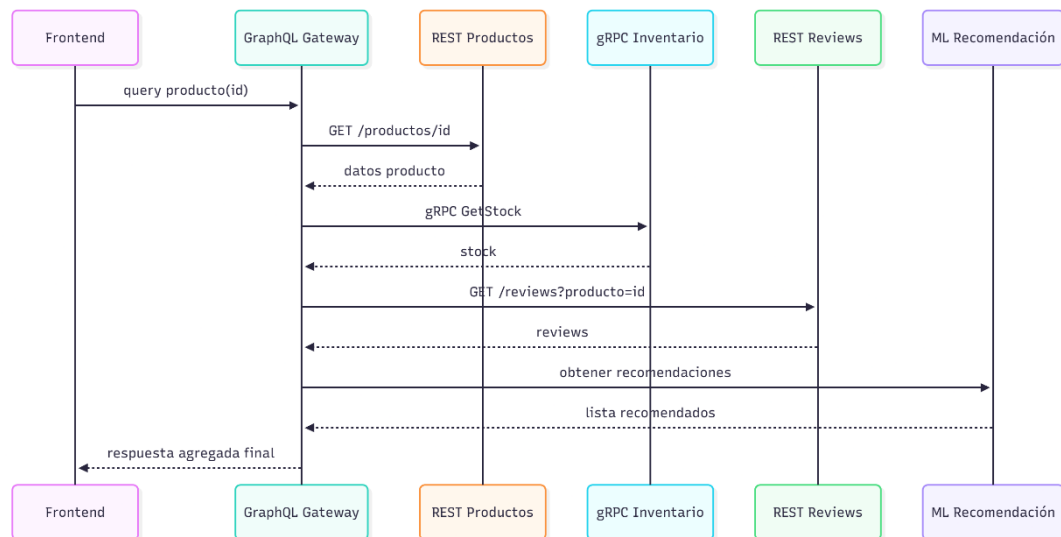
Query del frontend:

```
query ObtenerProductoCompleto($id: ID!) {  
  producto(id:$id){  
    id  
    nombre  
    precio  
    stock  
    reviews { comentario rating }  
    recomendados { id nombre precio }  
  }  
}
```

Cómo lo resuelve el GraphQL Gateway:

- producto: REST → /productos/{id}
- stock: gRPC → GetStock
- reviews: REST → /reviews?producto={id}
- recomendados: ML → Servicio IA interno

Figura 18. Integración híbrida completa



4.5.8. Ventajas del enfoque híbrido

1. **Reduce la deuda técnica.** No hay obligación de migrar REST a GraphQL ni viceversa.
2. **Optimiza el frontend.** Menos llamadas → menos latencia → mejor UX.
3. **Mejora la evolución del backend.** Microservicios pueden seguir siendo REST/gRPC según convenga.
4. **Permite transición progresiva.** Muy importante en organizaciones grandes o en proyectos existentes.
5. **Facilita la federación (Apollo Federation).** GraphQL unifica dominios y reduce acoplamiento.

4.5.9. Retos del enfoque híbrido

1. **Duplicación de modelos si no se gestiona bien.** Solución: *shared contracts* y codegen.

2. **Complejidad en el Gateway.** Resolver múltiples fuentes puede generar latencia.
3. **Seguridad multifactor.** REST usa ACL por endpoint; GraphQL usa ACL por campo.
4. **Monitoreo distribuido.** GraphQL requiere tracing profundo: resolver → microservicio.

4.5.10. Conclusión

La integración híbrida REST + GraphQL es una de las arquitecturas más efectivas para la construcción de sistemas escalables modernos.

Combina:

- la estabilidad y predictibilidad de REST,
- la flexibilidad y eficiencia de GraphQL,
- la capacidad de agregación del gateway,
- y la modularidad de los microservicios.

Para el prototipo tipo Amazon del curso, este enfoque habilita un desarrollo altamente profesional, alineado con arquitecturas reales usadas en la industria global.

Referencias citadas en la Clase 4.

- Apollo GraphQL. (2024). *Apollo Federation: Distributed Graph Architecture*. <https://www.apollographql.com>
- Facebook Engineering. (2015). *GraphQL: A Data Query Language*. <https://engineering.fb.com>
- Evans, E. (2020). *Domain-Driven Design Reference*. Addison-Wesley.
- AWS. (2024). *Building Scalable APIs with GraphQL and REST*.
- Netflix Tech Blog. (2024). *Federated Graph Architecture at Scale*.
- Shopify Engineering. (2023). *How Shopify Uses GraphQL for Its Admin Platform*.
- Google Cloud. (2024). *API Modernization Strategies*..

Definición de términos

- **GraphQL:** Lenguaje de consulta y runtime para APIs basado en un esquema fuertemente tipado.
- **Query:** Operación utilizada para leer datos en GraphQL.
- **Mutation:** Operación utilizada para modificar datos.
- **Subscription:** Mecanismo de actualización en tiempo real mediante WebSockets.
- **Schema (SDL):** Contrato tipado que define entidades, relaciones y operaciones GraphQL.
- **Resolver:** Función que obtiene el valor real de un campo del esquema.
- **Federation:** Modelo distribuido donde múltiples subgrafos se combinan en un supergraph unificado.
- **Supergraph:** Esquema completo resultante de combinar subgrafos mediante Apollo Federation.
- **Subgraph:** Esquema parcial mantenido por cada microservicio o dominio.
- **Gateway GraphQL:** Capa de orquestación que une subgrafos y ejecuta resoluciones distribuidas.

Recursos de profundización

Recurso: *Profundizacion1_clase4.docx*

ACTIVIDAD DE EVALUACIÓN:

INSTRUCCIONES	Responde las siguientes preguntas de opción múltiple relacionadas con los temas abordados en la Clase. Selecciona la alternativa correcta en cada caso. Al finalizar, revisa la retroalimentación para reforzar tu comprensión.				
Titulo	GraphQL y federación de servicios				
Dificultad		Baja	X	Media	Alta
RDAs Evaluados:	X	RDA 1			
		RDA 2			
		RDA 3			
Secciones Evaluadas		Clase: 4	GraphQL y federación de servicios		
PREGUNTA 1	¿Cuál es una ventaja clave de GraphQL frente a REST?				
CORRECTA	Permite obtener exactamente los campos solicitados mediante consultas declarativas.				
Incorrecta	Reduce completamente la necesidad de seguridad.				
Incorrecta	Reemplaza automáticamente microservicios REST.				
Incorrecta	No requiere esquema para funcionar.				
Retroalimentación de las respuestas	GraphQL evita el overfetching y underfetching al permitir que el cliente defina la forma exacta de la respuesta.				
PREGUNTA 2	¿Qué componente implementa la lógica que obtiene los datos de un campo del esquema GraphQL?				
CORRECTA	Resolver				
Incorrecta	Endpoint				
Incorrecta	DTO				
Incorrecta	Evento Webhook.				
Retroalimentación de las respuestas	Los resolvers vinculan el esquema con los microservicios REST, gRPC o bases de datos.				
PREGUNTA 3	¿Qué permite Apollo Federation en arquitecturas GraphQL distribuidas?				
CORRECTA	Combinar múltiples subgrafos en un supergraph unificado.				
Incorrecta	Reemplazar completamente las bases de datos internas.				
Incorrecta	Ejecutar consultas sin un esquema tipado.				
Incorrecta	Limitar GraphQL a un solo microservicio.				
Retroalimentación de las respuestas	La federación facilita la escalabilidad organizacional permitiendo que cada equipo mantenga su propio esquema.				
PREGUNTA 4	¿Cuál es el rol principal del GraphQL Gateway en una arquitectura federada?				
CORRECTA	Orquestar la composición de subgrafos y ejecutar resoluciones distribuidas.				
Incorrecta	Servir exclusivamente archivos estáticos del frontend.				
Incorrecta	Actuar como base de datos central.				
Incorrecta	Eliminar la necesidad de resolvers en cada servicio.				

Retroalimentación de las respuestas	El gateway unifica todos los subgrafos y se encarga de delegar resoluciones a los servicios correspondientes.
--	---

Enlaces externos recomendados

Título del enlace relacionado:

Apollo Federation – Official Documentation

Descripción: Guía oficial para implementar federación distribuida, composición de subgrafos y configuración de gateways GraphQL empresariales.

Enlace:

<https://www.apollographql.com/docs/federation>

Título del enlace relacionado:

GraphQL Specification

Descripción: Especificación formal del lenguaje GraphQL, incluyendo tipos, operaciones y reglas de ejecución.

Enlace:

<https://spec.graphql.org>

Título del enlace relacionado:

Shopify Engineering – How Shopify Uses GraphQL

Descripción: Caso de estudio real sobre cómo Shopify escaló a millones de consultas con GraphQL federado.

Enlace:

<https://shopify.engineering>

Título del enlace relacionado:

Netflix Tech Blog – Federated Graph Architecture

Descripción: Análisis profundo de la arquitectura federada usada por Netflix para miles de servicios internos.

Enlace:

<https://netflixtechblog.com/unlocking-dynamic-pages-the-evolution-of-netflixs-client-server-graphql-apis-8b4631a59b39>